(d) (2 points) External fragmentation

(e) (2 points) Demand paging

國立臺灣大學 期中/期末 考試答案卷

National Taiwan University Midterm/Final Examination Answer Sheet

記分 Score	教師簽名或蓋章 Lecturer's signature
90	

課程編號 Course no.

科目 Course title 作業系統 學院 College 管理 學系 Department 資訊管理 組 Division 二 年級 Year

考試日期 Date 2011 年 Y 4 月 M 19 日 D 學號 Student ID no. B98705046 姓名 Name 邊翔

從此處開始寫起。試卷用紙務須節用。非經主試認可不得續用其他紙張作答。

Please write from here.

記分欄

1. (a) The CPU has to be constantly running a loop to see if busy() implies that an event has occured for busy-waiting. Interrupt-based operations notifies the CPU only when an event has actually occured, so the CPU doesn't have to waste time and power running the loop.

(b) Hardware interrupt happens when a hardware device has data for CPU. (ex. hard-disk data are available) The device controller loads the data into the device's registers first. After the data is ready, it sends a signal via the interrupt-request line to tell the CPU that the data is available. After the CPU receives the interrupt, it stops whatever it is doing at the moment and uses state saving to temporarily save what it is doing. Then the CPU checks the interrupt vector to see what the sender of the interrupt wants to do and what device sent it. The interrupt-specific handler then deals with the interrupt. After the requested work is done, the CPU resumes whatever it was doing before the interrupt

data is available. After the CPU receives the interrupt, it stops whatever it is doing at the moment and uses state saving to temporarily save what it is doing. Then the CPU checks the interrupt vector to see what the sender of the interrupt wants to do and what device sent it. The interrupt-specific handler then deals with the interrupt. After the requested work is done, the CPU resumes whatever it was doing before the interrupt.

5 (c) The CPU can identify what the interrupt needs quickly by checking the interrupt vector.

2. Multiprogramming means having the capability to run multiple programs or tasks during the same period of time without needing the first one to finish. Timesharing means switching between running programs so fast as though they are running at the same time.

3 multiprogramming 在遇到 interupt 時 CPU 先換到下個 process, 讓 CPU 一直處於 busy state.

first, save all system call index into register

12 3. Most systems run under dual-mode — user mode and kernel mode. When the user uses an specific API function to make a specific system call under user mode, the API function sends out a software interrupt to the CPU, along with a system call number. After the CPU receives the interrupt, it switches to kernel mode and then checks the table of code pointers to find which system call the interrupt is requesting using the system call number. After the requested system call is executed, the CPU returns control to the user mode.

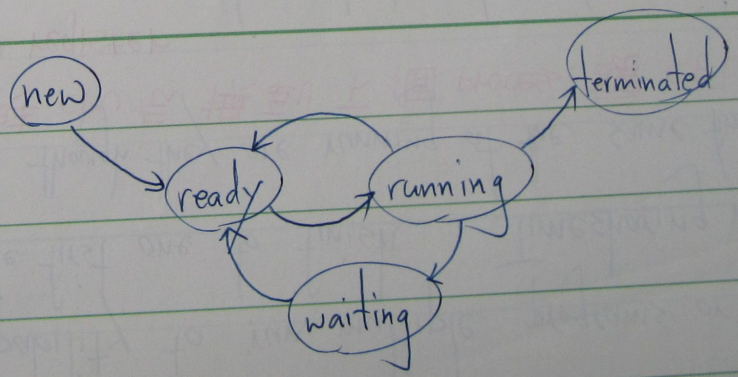
(a) (2 points) Turnaround time
(b) (2 points) Race condition
(c) (2 points) Deadlock
(d) (2 points) Fut...

Total: 100 points

記分欄 轉頁從此開始寫起。 第二頁

(10) 4. Layered operating system structure is easier to use for the users because users can use high-level language or simpler instructions to do what they want to do at layer N. Also, layered operating system structure is easier to debug because programmers can start debugging at Layer 0 (kernel) then move their way up. If a bug is encountered at a certain layer, then the bug must be in that layer. However, it is sometimes difficult to implement because it is difficult to separate different layers apart. (some tasks might be ambiguous) What's more, a layered operating system structure may require several repeated calls all the way from Layer N to Layer 0 when the user makes a system call request.

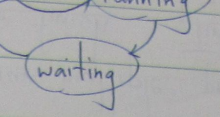
5. (a) Ready.
(15) (b) Running.
(c) Waiting.
(d) Ready.
(e) Ready.



int main(int argc, char *argv[])

(d) Ready

(e) Ready



15

6.
```
int main(int argc, char *argv[])
{  int N= atoi(argv[1]);
   pid_t pid;
   int segment_id;
   int *shared_memory;
   segment_id=shmget(IPC_PRIVATE, size, S_IRUSR | S_IWUSR);
   shared_memory = shmat(segment_id, NULL, 0);
   for(int i=1; i<=N; i++)
   {  pid = fork();
      if(pid < 0){
         fprintf(stderr, "Fork Failed");
         exit(-1);
      } else if (pid == 0){   //child part
         if(i==1){
            shared_memory = 1;
         } else {
            *shared_memory = *shared_memory * i;
         }
         shmdt(shared_memory);
         exit();
      } else {   // parent part
         wait();
      }
   }
}
```

shared_memory 裡只有 N!

題目要 ΣN!

```
    printf("the result is %d\n", *shared_memory);
    shmdt(shared_memory);
    shmctl(shared_memory, IPC_RMID, NULL);
    return 0;
}
```

5

7. (a) Many-to-many thread model allocates a number of user threads to a number of kernel threads. But when one user thread is in state waiting, the kernel thread that it is currently occupying is also waiting, while the kernel thread can be used more efficiently if it uses the waiting time to run other threads.

10

(b) When a user thread becomes waiting, the upcall handler first allocates a new virtual kernel thread for itself. Then the upcall handler records which user thread is in waiting and why. After that, it detaches the waiting user thread from the kernel thread, then allocates the new kernel thread to another user thread to run. After the waiting user thread is ready to run again, the upcall handler takes back the kernel thread and gives it back to the original user thread for running.

15